

⚡ ADVANCED COMPILER ENGINEERING

LLVM Kaleidoscope

Compiler Architecture & Implementation

Comprehensive Deep-Dive into Token Rules, Quadruple/Triple IR, AST Design, Error Handling, Memory Segments, and Target Architecture

📁 **Repository:** [llvm-kaleidoscope](#)

⚙️ **Backend:** LLVM IRBuilder & Target Machine

💻 **Language:** C++14 / Modern OOP

🎯 **Focus:** Frontend, IR Rules & Output Segments

Project Objectives

01

1. Hand-Written Lexing & Operator-Precedence Parsing

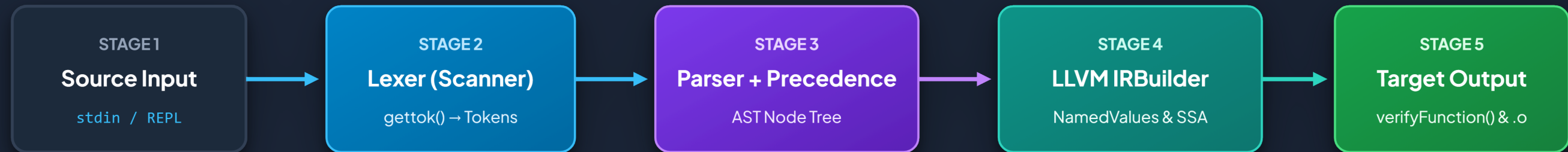
02

2. Polymorphic AST Construction & Scoped Semantic Analysis

03

3. SSA LLVM IR Generation & Target Architecture Emission

High-Level Compiler Pipeline



🔧 Interactive REPL Loop

Reads standard input continuously, classifies statements into definitions, externs, or immediate top-level expressions.

🛡️ Verification & Segment Lowering

Emits validated LLVM byte-code into Module sections and dumps diagnostic text to standard output/error channels.

Complete Token Types & Lexer Rules

Complete Language Token Mapping

Token Category	Enum Identifier	Value	Lexer Match Pattern / Keyword
EOF / Stream	tok_eof	-1	End-of-File stream (EOF)
Function Defs	tok_def, tok_extern	-2, -3	Keywords "def", "extern"
Primary Entities	tok_identifier, tok_number	-4, -5	[a-zA-Z0-9]+, [0-9.]+ (→ double)
Control Flow	tok_if, tok_then, tok_else	-6, -7, -8	Keywords "if", "then", "else"
Loop Constructs	tok_for, tok_in	-9, -10	Keywords "for", "in"
Custom Operators	tok_binary, tok_unary	-11, -12	Keywords "binary", "unary"
Mutable Variables	tok_var	-13	Keyword "var"
Operators & Delims	ASCII direct	[0-255]	+ - * / < > = () { } , ;

```
// Complete Token Definitions in lexer/token.h
enum Token {
    tok_eof = -1,
    // Commands & Definitions
    tok_def = -2, tok_extern = -3,
    // Primary Entities
    tok_identifier = -4, tok_number = -5,
    // Control Flow & Loops
    tok_if = -6, tok_then = -7, tok_else = -8,
    tok_for = -9, tok_in = -10,
    // User-Defined Operators & Variables
    tok_binary = -11, tok_unary = -12, tok_var = -13
};

// Lexical Scanner Match Dispatch in lexer/lexer.cpp
if (IdentifierStr == "def") return tok_def;
if (IdentifierStr == "extern") return tok_extern;
if (IdentifierStr == "if") return tok_if;
if (IdentifierStr == "then") return tok_then;
if (IdentifierStr == "else") return tok_else;
if (IdentifierStr == "for") return tok_for;
if (IdentifierStr == "in") return tok_in;
if (IdentifierStr == "var") return tok_var;
return tok_identifier;
```

Quadruple Rule in Intermediate Code Generation

4 Quadruple Structure: (op, arg1, arg2, result)

Our compiler strictly adopts the **Quadruple representation** where every intermediate operation explicitly specifies four components:

- **op**: The opcode / LLVM instruction type (e.g. fadd, fmul, fcmp, call).
- **arg1**: First input operand (variable, constant, or SSA register).
- **arg2**: Second input operand (or null for unary/return).
- **result**: Explicit named destination register (e.g. %addtmp, %multmp).

Why Quadruples in Modern LLVM?

Explicit SSA naming ensures compiler optimization passes (GVN, Dead Code Elimination) can safely reorder instructions without breaking statement index pointers.

⚡ Compiler Quadruple Instruction Set

Operation	Quadruple 4-Tuple	LLVM IRBuilder API Call
Addition	(fadd, L, R, %addtmp)	Builder.CreateFAdd(L, R, "addtmp")
Subtraction	(fsub, L, R, %subtmp)	Builder.CreateFSub(L, R, "subtmp")
Multiplication	(fmul, L, R, %multmp)	Builder.CreateFMul(L, R, "multmp")
Comparison	(fcmp_ult, L, R, %cmptmp)	Builder.CreateFCmpULT(L, R, "cmptmp")
Type Cast	(uitofp, %cmptmp, double, %booltmp)	Builder.CreateUIToFP(L, Type, "booltmp")
Function Call	(call, @func, [Args], %calltmp)	Builder.CreateCall(CalleeF, ArgsV, "calltmp")
Return	(ret, %val, -, -)	Builder.CreateRet(RetVal)

Parser Architecture & Operator Precedence



Operator Precedence Climbing

Uses recursive descent combined with operator precedence climbing for mathematical expressions.

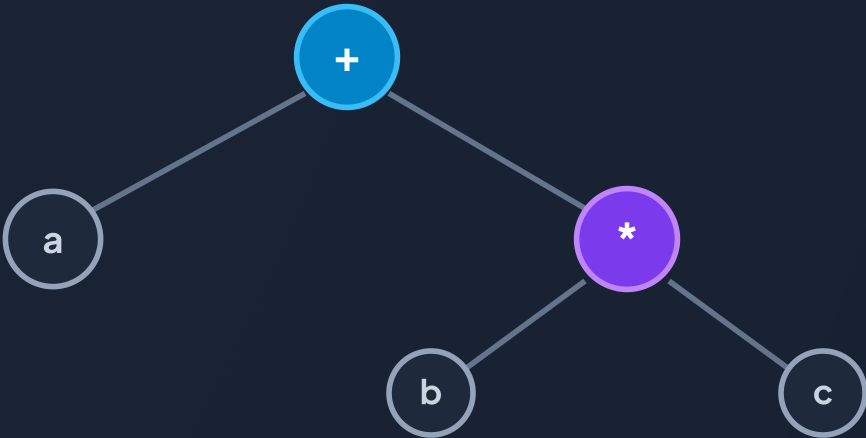
Precedence Table

<:10 +, -:20 *:40

- ParsePrimary() resolves numbers, variables, function calls, and parenthesized expressions.
- ParseBinOpRHS() dynamically groups subtrees based on comparative token precedence.



Expression Tree: $a + b * c$



Operator precedence ensures * binds tighter, forming the RHS subtree first.

Abstract Syntax Tree (AST) Class Hierarchy

1 NumberExprAST

Numeric constants (e.g. 42.0).

```
double Val;  
codegen();
```

2 VariableExprAST

Variable reference (e.g. x).

```
string Name;  
codegen();
```

3 BinaryExprAST

Binary operations (LHS op RHS).

```
char Op;  
unique_ptr LHS, RHS;
```

4 CallExprAST

Function calls: sin(x, y).

```
string Callee;  
vector Args;
```

5 PrototypeAST

Captures function metadata: signature name and parameter identifiers list. Emits `llvm::Function*` declarations.

6 FunctionAST

Combines a `PrototypeAST` with a body `ExprAST`. Manages the function entry basic block and populates argument symbol tables.

Error Handling Based on All Token Rules

Token Type / Rule	Trigger Condition	Error Diagnostic Emitted	Compiler Recovery Action
tok_identifier (Prototype)	Missing function name after def/extern	"Expected function name in prototype"	Returns nullptr via LogErrorP
' (' (Prototype Open)	Missing opening parenthesis in parameter list	"Expected '(' in prototype"	Returns nullptr via LogErrorP
') ' (Prototype Close)	Missing closing parenthesis after parameters	"Expected ')' in prototype"	Returns nullptr via LogErrorP
') ' (Paren Expression)	Unmatched closing parenthesis in (expr)	"Expected)"	Returns nullptr via LogError
', ' / ') ' (Call Args)	Missing comma or closing paren in argument list	"Expected ')' or ', ' in argument list"	Aborts call node construction
tok_identifier (Scope)	Identifier not found in NamedValues symbol table	"Unknown variable name"	Returns nullptr via LogErrorV
Function Arity Rule	Argument count $\neq$ prototype parameter count	"Incorrect # arguments passed"	Aborts function call codegen
Codegen Failure Rule	Function body codegen returns nullptr	Transactional Module Rollback	TheFunction->eraseFromParent()

Output Channels & Target Memory Segments



1. Standard Output Streams

- **Standard Error (stderr):**
 - REPL interactive prompt: `ready>`
 - Diagnostics & Error Traps: `LogError: ...`
 - IR Emission feedback: `Read function definition:`
 - Final Module Dump: `TheModule->print(errs(), nullptr)`
- **Standard Input (stdin):**
 - Interactive command stream & file pipe execution (`./main < test.kld`)



2. Target Memory Segments

- **Text Segment (.text / Code):** Generated machine instructions for function basic blocks (e.g. `@foo, @__anon_expr`).
- **Read-Only Data (.rodata / Constant Pool):** Floating-point constants emitted via `ConstantFP::get(TheContext, APFloat(Val))`.
- **Symbol Table (.symtab / Relocations):** External declarations (`declare double @sin(double)`) and local symbol definitions.
- **Heap & Memory Arena:** AST ownership nodes managed via `std::unique_ptr` and LLVM `IRContext` allocation pools.

Target Architecture & Native Binary Emission



Target Triples

Defines cross-platform target hardware configurations:

```
"x86_64-pc-linux-gnu"  
"x86_64-w64-windows-msvc"  
"aarch64-apple-darwin"
```

- `InitializeAllTargetInfos()`
- `InitializeAllTargets()`
- `InitializeAllAsmPrinters()`



TargetMachine & ABI

Configures CPU capabilities, ABI conventions, and memory alignments:

- **DataLayout:** Dictates 64-bit pointer size and memory alignments.
- **CPU Architecture:** Targets specific CPUs (e.g. generic, haswell).
- **Relocation Model:** Configured for Position Independent Code (`Reloc::PIC_`).



Object File Emission

Lowers LLVM IR directly into binary object files (`.o` / `.obj`):

```
legacy::PassManager pass;  
auto FileType = CGFT_ObjectFile;  
TheTargetMachine→  
    addPassesToEmitFile(pass, dest, nullptr, FileType);  
pass.run(*TheModule);
```

Linkable via `clang/gcc/ld` into native executables.

Execution Trace & Output Segment Breakdown

1. Tokens & Grammar

```
[tok_def] "def"  
[tok_identifier] "foo"  
['(']  
[tok_identifier] "x"  
[tok_identifier] "y"  
[')']  
[tok_identifier] "x"  
['+']  
[tok_identifier] "y"  
['*']  
[tok_number] 4.0  
[';']
```

2. Quadruple / AST Lowering

```
// Quadruple Form  
(0) (FMUL, y, 4.0, %multmp)  
(1) (FADD, x, %multmp, %addtmp)  
(2) (RET, %addtmp, -, -)  
  
// Scoping Check  
NamedValues["x"] = %x  
NamedValues["y"] = %y
```

3. Emitted Output & Segment

```
// Channel: stderr  
Read function definition:  
// Segment: .text (Code)  
define double @foo(double %x, double %y) {  
entry:  
    %multmp = fmul double %y, 4.000000e+00  
    %addtmp = fadd double %x, %multmp  
    ret double %addtmp  
}
```

Future Work Plan & Implementation Milestones

P1 Phase 1: JIT & Optimization
Milestone 1 • Chapter 4

- Integrate **LLVM OrcJIT / LLJIT** to evaluate top-level expressions immediately.
- Attach `FunctionPassManager` with GVN and Instruction Combining passes.

P2 Phase 2: Control Flow & State
Milestone 2 • Chapters 5–7

- Implement `IfExprAST` with conditional phi-nodes.
- Add `ForExprAST` loop construct.
- Add mutable local variables with `stack allca` and LLVM `mem2reg`.

P3 Phase 3: Target Object Emission
Milestone 3 • Chapters 8–9

- Configure target machine triple and data layouts.
- Emit native object code (`.o` / `.obj`) and link to standalone executables.
- Add DWARF / CodeView debug symbols.

 SUMMARY & WRAP-UP

Thank You! Questions & Discussion

Key Takeaways:

- Clear separation of concerns between Tokenizer, Grammar Parser, AST, and LLVM Backend.
- Comprehensive token-rule-based error handling ensuring graceful REPL recovery and transactional IR rollbacks.
- Production-grade lowering to verified SSA Quadruple IR and clear target architecture mapping for native binary compilation.

 **Reference:** llvm.org/docs/tutorial

 **Repository:** [llvm-kaleidoscope](https://github.com/llvm-kaleidoscope)